

第十四周项目报告

基于 README.md 自动生成

1. 项目概述

本项目基于 ROS2 与 turtlesim 仿真环境，实现了一个支持手机远程控制和自主探索迷宫的小乌龟机器人系统。

项目主要包含两个功能模块：

1. 手机遥控模块

- 通过手机浏览器访问控制网页；
- 利用局域网（Tailscale）与 ROS2 系统通信；
- 实现前进、后退、左转、右转等实时控制。

2. 自动探索模块

- 基于经典巡墙算法（Wall Following）；
- 采用右手法则（Right-Hand Rule）进行路径搜索；
- 在迷宫环境中实现自主导航与探索。

系统整体架构如下：

```
手机浏览器
↓
网页控制界面（HTML/JS）
↓
HTTP / WebSocket
↓
Python Bridge Server
↓
ROS2 Node
↓
/turtle1/cmd_vel
↓
turtlesim
```

这是小乌龟 效果见截图



这是效果图

2. 系统总体设计

系统采用前后端分离架构，由以下三个部分组成：

2.1 前端控制层

负责提供手机端操作界面。

主要功能：

- 用户输入控制指令；
- 模式切换（手动 / 自动）；
- 通过网络向服务器发送控制请求。

2.2 桥接通信层

负责连接网页与 ROS2。

主要功能：

- 接收网页请求；
- 解析控制指令；
- 转换为 ROS2 速度控制消息；
- 发布至小乌龟运动控制话题。

2.3 ROS2 控制层

负责控制 turtlesim 中的小乌龟运动。

主要功能：

- 接收速度指令；
- 执行运动控制；
- 实现自动探索算法。

3. 手机遥控与 ROS2 通信模块实现

3.1 手机遥控器网页前端（HTML/JavaScript）

在 `week14_starters/turtlesim_remote/frontend/` 目录下开发了一个轻量级网页控制界面，用于通过手机远程操控小乌龟运动。

界面设计

网页提供以下控制组件：

- 前进（Forward）
- 后退（Backward）
- 左转（Turn Left）
- 右转（Turn Right）
- 手动遥控 / 自动探索模式切换按钮

用户通过手机浏览器访问网页即可实现实时控制。

通信逻辑

前端采用 JavaScript 实现控制指令发送。

当按钮被点击或长按时：

```
direction=forward  
direction=backward  
direction=left  
direction=right
```

模式切换时：

```
mode>manual  
mode=auto
```

利用 Fetch API 或 WebSocket 将请求发送至桥接服务器。

3.2 Python 桥接程序与局域网通信

桥接程序同时充当：

- Web 服务器
- ROS2 节点

网络服务端

监听地址：

```
0.0.0.0:8080
```

采用 Flask 或 FastAPI 接收来自手机端的请求。

ROS2 控制节点

桥接节点继承：

```
rcipy.node.Node
```

创建速度发布者：

```
geometry_msgs/msg/Twist
```

并向以下话题发布控制消息：

```
/turtle1/cmd_vel
```

控制流程

```
手机网页
↓
HTTP/WebSocket
↓
Bridge Server
↓
ROS2 Publisher
↓
/turtle1/cmd_vel
↓
turtlesim
```

4. 自动探索迷宫算法实现

自动探索功能属于本项目核心内容。

项目基于参考文件 `explorer.py` 进行了二次开发，实现了适用于 `turtlesim` 环境的自主巡墙算法。

4.1 算法原理：右手法则（Right-Hand Rule）

右手法则是一种经典迷宫搜索方法。

其基本思想为：

始终保持机器人右侧贴着墙壁前进，在连通迷宫中最终能够找到出口。

由于 turtlesim 不提供激光雷达等传感器，本项目利用：

- 边界检测
- 碰撞模拟
- 位姿信息

来判断周围墙体状态。

状态感知

系统维护两个核心状态：

`front_blocked`

表示前方是否存在障碍物。

`right_open`

表示右侧是否存在可通行空间。

巡墙状态机

系统根据环境状态执行不同动作。

状态 A：寻找墙壁

行为：

- 持续向前移动；
- 直到检测到墙壁。

状态 B：沿墙直行

条件：

- 前方无障碍；
- 右侧存在墙壁。

行为：

- 保持匀速前进；

- 维持与墙壁的接触。

状态 C：右转弯

条件：

- 右侧突然变为空旷区域。

行为：

- 判定为外侧拐角；
- 向右转向；
- 重新贴近墙壁。

状态 D：左转弯

条件：

- 前方检测到障碍物。

行为：

- 停止前进；
- 原地向左旋转；
- 直到前方重新畅通。

4.2 核心控制逻辑

自动探索控制循环如下：

```
def p_control_loop(self):  
  
    if self.front_blocked:  
  
        # 前方有墙，执行左转  
        self.msg.linear.x = 0.0  
        self.msg.angular.z = 1.5  
  
    elif self.right_open:  
  
        # 右侧出现空旷区域  
        self.msg.linear.x = 1.0  
        self.msg.angular.z = -1.0  
  
    else:  
  
        # 正常沿墙前进  
        self.msg.linear.x = 1.5  
        self.msg.angular.z = 0.0  
  
    self.publisher_.publish(self.msg)
```

控制策略分析

环境状态	线速度	角速度	执行动作
前方有障碍	0.0	+1.5	原地左转
右侧空旷	1.0	-1.0	向右贴墙
正常巡墙	1.5	0.0	直线前进

该策略具有实现简单、计算开销小、稳定性高等特点，能够满足课程项目对于自主探索迷宫功能的要求。

5. 项目测试与结果分析

5.1 手动遥控测试

测试内容：

- 前进控制
- 后退控制
- 左转控制
- 右转控制
- 模式切换控制

测试结果：

- 所有控制指令均能够正确发送；
- ROS2 节点能够实时接收消息；
- 小乌龟运动响应流畅；
- 局域网通信稳定。

5.2 自动探索测试

测试内容：

- 墙体跟踪能力；
- 转角识别能力；
- 死胡同处理能力；
- 连续探索能力。

测试结果：

-
- 能够稳定执行右手法则；
 - 可自动绕过障碍区域；
 - 能够完成连续巡墙探索；
 - 未出现明显卡死现象。

6. 总结与展望

本项目完成了基于 ROS2 的手机遥控与自主探索系统设计，实现了网页控制、网络通信以及自动巡墙导航等功能。

项目成果包括：

- 手机网页远程控制；
- ROS2 通信桥接；
- turtlesim 运动控制；
- 基于右手法则的自主探索算法。

未来可进一步扩展：

- 引入激光雷达模拟；
- 使用 SLAM 建图技术；
- 实现最短路径规划；
- 支持多机器人协同探索；
- 接入真实移动机器人平台。

通过本项目，深入掌握了 ROS2 节点通信机制、网络控制系统设计以及自主导航算法的基本实现方法。